Take-Home Assignment: Web Deployed Rich Text Editor

Role: AI-Native Full Stack Software Engineer
Estimated effort: 6–10 hours (hard cap: 12 hours)
Submission deadline: 7 calendar days from receipt

Context

You are building a web-deployed rich text editor aimed at professionals who need a simple, fast writing workspace. Your task is to build a small but complete product slice: a marketing site that converts visitors into paid subscribers, plus the authenticated editor experience they unlock after paying.
This assignment is designed to evaluate how you scope, ship, and reason about Product, not whether you can memorize every API. You should use AI assistants. We care about judgment, tradeoffs, security, and clarity of communication.

Product Summary

Build a single web application with two surfaces:

1. Public marketing site that explains the product and drives signup and payment
2. Authenticated app that is a rich text document workspace for paying subscribers

Non-paying registered users may exist in the system, but document creation/editing is restricted to active subscribers.

Must-Have Requirements

1. Marketing & Conversion

Create a landing experience that includes:

- Clear value proposition (headline, subhead, 2 to 3 benefit bullets)
- Primary CTA: Start free trial or Subscribe (your choice, but be consistent)
- Pricing section with at least one paid plan
- Basic trust elements (e.g. feature list, FAQ, or social proof placeholder)
- Responsive layout (mobile and desktop)

Conversion goal: a visitor can understand the product, register, pay, and land in the editor in one coherent flow.

2. User Registration & Authentication

- Users can register with email/password (OAuth optional)
- Users can log in and log out
- Session handling must be server-validated (do not rely on client-only auth checks for protected actions)
- Show appropriate UI states: logged out, logged in (non-subscriber), logged in (subscriber)

3. Rich Text Document Management (Subscribers Only)

Paying subscribers can:

Action	Requirement
Create	New document with default title
Edit	Rich text content in a WYSIWYG or equivalent editor
Save	Persist content to your backend (auto-save or manual save is fine; document your choice)
Rename	Change document title
Delete	Remove document (confirm before delete)

Additional expectations:

- Documents are stored on your site (your database/filesystem—not only browser localStorage)
- Document list view (sidebar or dashboard) showing titles and last updated time
- Empty states and basic error handling (e.g. failed save)
- Use any rich text editor of your choice

4. Stripe Sandbox Payments

Integrate Stripe test mode so users can pay for access:

- At least one subscription or one-time purchase plan (subscription preferred)
- Checkout must use Stripe (Checkout Session or Payment Element—your choice)
- After successful payment, user gains subscriber access to documents
- Handle webhook confirmation server-side (do not grant access based only on client redirect)
- Expose a way to verify subscription status in your app (e.g. “Manage billing” link to Stripe Customer Portal is a bonus but not required)

Use Stripe test keys only. Do not commit secrets.

Technical Guidelines

Area	Guidance
Language	TypeScript encouraged for frontend and backend
Stack	Your choice (React, Next.js, SvelteKit, etc.)
Hosting	Self-hosted/local is acceptable; cloud deployment is not required
Cloud (if used)	Restrict to AWS or GCP only
Database	Your choice (SQLite is fine for this assignment)
Secrets	Environment variables + .env.example; no keys in repo

Out of Scope (Do Not Spend Time On)

- Production-grade infra (K8s, multi-region, CDN tuning)
- Team collaboration / real-time multi-user editing
- Mobile native apps
- Full admin panel
- Email verification
- Custom Stripe pricing UI beyond what's needed for the flow

If you skip something due to time, say so in your README.

Deliverables

Submit a Git repository (GitHub/GitLab link or zip) containing:

1. Runnable application

- README.md with setup steps (should work on a fresh machine in <15 minutes)
- .env.example listing all required variables
- Dependency install and start commands

2. README must include

- Architecture overview (1 short diagram or bullet flow is enough)
- Tradeoffs you made and what you'd do next with another day
- Stripe test flow (test card, how to trigger webhook locally e.g. Stripe CLI)
- Known limitations

3. Short write-up (about 1 page)

Answer:

1. How does your app decide a user is an active subscriber?
2. What happens if payment succeeds but the webhook is delayed?
3. One security decision you made and why

4. Demo

- Either a 2 to 4 minute Loom/video walkthrough, or screenshots/GIFs in the README covering: landing → register → pay → create/edit/save/rename/delete document

Evaluation Rubric

We score holistically across these areas:

Category	Weight	What we look for
Product completeness	30%	End-to-end flow works; subscriber gating is correct
Code quality	25%	Readable structure, sensible separation, TypeScript usage, runnable acceptance tests
Security & correctness	20%	Auth, webhook verification, server-side authorization
UX & polish	15%	Clear flows, empty/error states, reasonable marketing page
Communication	10%	README, tradeoffs, honest scope notes

AI-native signal: we value concise docs, good prompting of constraints in code comments where non-obvious, as well as evidence that you reviewed AI-generated code rather than pasted it blindly.

Acceptance Criteria Checklist

We will manually verify:

- Landing page communicates product and pricing
- User can register and log in
- Non-subscriber cannot create/edit documents (clear upgrade path)
- Stripe test payment unlocks subscriber features
- Webhook updates subscription state server-side
- Subscriber can create, edit, save, rename, and delete documents
- Documents persist across logout/login
- App runs locally from README instructions
- No secrets committed

Stripe Test Hints (for candidates)

- Use Stripe test cards (e.g. 4242 4242 4242 4242)
- For local webhooks: Stripe CLI stripe listen --forward-to localhost:...
- Store customer_id and/or subscription_id on your user record

Submission

Reply with:

1. Repository URL
2. Approximate time spent
3. Anything you'd ask us if this were a real sprint

FAQ

Can I use a template or starter? Yes. Credit it and explain what you changed.
Can I use AI tools? Yes. This role is AI-native. Tell us how you used them and what you verified manually.
Do I need to deploy? No. Local/self-hosted is fine.
Subscription vs one-time purchase? Subscription preferred; one-time access pass is acceptable if gating logic is clear.
Can registered users get a free tier? Optional. Minimum bar: paid users get full document features; non-paying users do not.